

Algorithm for file updates in Python

Project description

At my organization, access to restricted content is controlled with an allow list of IP addresses. The "allow_list.txt" file identifies these IP addresses. A separate remove list identifies IP addresses that should no longer have access to this content. I created an algorithm to automate updating the "allow_list.txt" file and remove these IP addresses that should no longer have access.

Open the file that contains the allow list

```
In [ ]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a List of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First Line of `with` statement
with open(import_file, "r") as file:
```

For the first part of the algorithm I used a variable called `import_file` and assigned the string with the file name "allow_list.txt" to it. Then I used a with statement `with open` to open the text file and the file variable `as file` to store the file while I work with it. The first parameter `import_file` is the name of the file I want to open and the second parameter "r" tells Python what I want to do with the file. In this case it is "r" so that means we want to read the file.

Read the file contents

```
In [5]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a List of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()
```

In order to read the file contents, I used the `.read()` method to convert it into string format. When using an `.open()` function that includes the argument "r" for read, I can call the `.read()` function in the body of the with statement. I applied the `.read()` method to the

file variable identified in the with statement and stored this string in a variable called `ip_addresses`.

Convert the string into a list

```
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()
# Use `.split()` to convert `ip_addresses` from a string to a List
ip_addresses = ip_addresses.split()
```

The file is now readable, but I also need to be able to remove individual ip addresses from the list that could be in the `remove_list`. To do this I need to convert the string into a list. I do this by using the `.split()` method on the variable `ip_addresses`. Before the `.split()` method each ip address is on a new line, this new line is treated as a separation point in Python. So after using the `.split()` method there is now a comma separating each ip address and the data is in list format.

Iterate through the remove list

```
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()
# Use `.split()` to convert `ip_addresses` from a string to a List
ip_addresses = ip_addresses.split()
# Build iterative statement
# Name Loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:
```

The second list called `remove_list` contains all of the ip addresses that should be removed from the `ip_addresses` list. To do this I had to use an iterative statement containing a for loop. The overall purpose of a `for` loop in a Python algorithm like this is to apply specific code statements to all elements in a sequence. The `for` keyword starts the for loop and is followed by the loop variable `element` and then the keyword `in`. The keyword `in` indicates to iterate through `ip_addresses` and assign each value to the loop variable `element`.

Remove IP addresses that are on the remove list

```
# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name Loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)
```

Now that the for loop will iterate through the elements in `ip_addresses` I needed to create a conditional statement that will remove the ip addresses that appear in the `remove_list` from `ip_addresses`. To do this I made a conditional statement `if element in remove_list:`, and within that `ip_addresses.remove(element)`. The `if` and `in` keywords are used to set conditions on what to look for in the `ip_addresses` list as it's being iterated. It's saying that when an element is found in `ip_addresses` that is also in `remove_list` remove it, which is what the `.remove()` method does. The `.remove()` method is attached to `ip_addresses` because that is the list elements will be removed from and `element` is the argument of the `.remove()` method that will be removed, so it is in parenthesis. This works because there are no duplicates in the `ip_addresses` list. The `.remove()` method removes only the first occurrence of a value.

Update the file with the revised list of IP addresses

```
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

As a final step in my algorithm, I needed to update the allow list file with the revised list of IP addresses. To do so, I first needed to convert the list back into a string. I used the `.join()` method for this. The `.join()` method combines all items in an iterable into a string. The `.join()` method is applied to a string containing characters that will separate the elements in

the iterable once joined into a string. In this algorithm, I used the `.join()` method to create a string from the list `ip_addresses` so that I could pass it in as an argument to the `.write()` method when writing to the file `"allow_list.txt"`. I used the string `("\n")` as the separator to instruct Python to place each element on a new line.

Then, I used another `with` statement and the `.write()` method to update the file. This time, I used a second argument of `"w"` with the `open()` function in my `with` statement. This argument indicates that I want to open a file to write over its contents. When using this argument `"w"`, I can call the `.write()` function in the body of the `with` statement. The `.write()` function writes string data to a specified file and replaces any existing file content. In this case I wanted to write the updated allow list as a string to the file `"allow_list.txt"`. This way, the restricted content will no longer be accessible to any IP addresses that were removed from the allow list. To rewrite the file, I appended the `.write()` function to the file object `file` that I identified in the `with` statement. I passed in the `ip_addresses` variable as the argument to specify that the contents of the file specified in the `with` statement should be replaced with the data in this variable.

Summary

I created an algorithm that removes IP addresses identified in a `remove_list` variable from the `"allow_list.txt"` file of approved IP addresses. This algorithm involved opening the file, converting it to a string to be read, and then converting this string to a list stored in the variable `ip_addresses`. I then iterated through the IP addresses in `remove_list`. With each iteration, I evaluated if the element was part of the `ip_addresses` list. If it was, I applied the `.remove()` method to it to remove the element from `ip_addresses`. After this, I used the `.join()` method to convert the `ip_addresses` back into a string so that I could write over the contents of the `"allow_list.txt"` file with the revised list of IP addresses.